

Currency Converter API

Objective

Design and implement a robust, scalable, and maintainable currency conversion API using C# and ASP.NET Core, ensuring high performance, security, and resilience.

Requirements

1. Endpoints

1.1 Retrieve Latest Exchange Rates

- Fetch the latest exchange rates for a specific base currency (e.g., EUR).

1.2 Currency Conversion

- Convert amounts between different currencies.
- Exclude TRY, PLN, THB, and MXN from the response and return a bad request if these currencies are involved.

1.3 Historical Exchange Rates with Pagination

- Retrieve historical exchange rates for a given period with pagination (e.g., 2020-01-01 to 2020-01-31, base EUR).

2. API Architecture & Design Considerations

2.1 Resilience & Performance

- Implement **caching** to minimize direct calls to the Frankfurter API.
- Use **retry policies with exponential backoff** to handle intermittent API failures.
- Introduce a **circuit breaker** to gracefully handle API outages.

2.2 Extensibility & Maintainability

- Implement **dependency injection** for service abstractions.
- Design a **factory pattern** to dynamically select the currency provider based on the request.
- Allow for future integration with multiple exchange rate providers.

2.3 Security & Access Control

- Implement **JWT authentication**.
- Enforce **role-based access control (RBAC)** for API endpoints.
- Implement **API throttling** to prevent abuse.

2.4 Logging & Monitoring

- Log the following details for each request:
 - Client IP
 - ClientId from the JWT token
 - HTTP Method & Target Endpoint
 - Response Code & Response Time
- Correlate requests against the Frankfurter API for debugging.
- Use **structured logging** (e.g., **Serilog with Seq or ELK stack**).
- Implement **distributed tracing** (e.g., **OpenTelemetry**).

3. Testing & Quality Assurance

- Achieve **90%+ unit test coverage**.
- Implement **integration tests** to verify API interactions.
- Provide test coverage reports.

4. Deployment & Scalability Considerations

- Ensure the API supports deployment in **multiple environments (Dev, Test, Prod)**.
- Support **horizontal scaling** for handling large request volumes.
- Implement **API versioning** for future-proofing.

Evaluation Criteria

Candidates will be evaluated on:

- ✓ **Solution Architecture** (Extensibility, Maintainability, Resilience)
- ✓ **Code Quality & Best Practices** (SOLID, Design Patterns, DI)
- ✓ **Security & Performance** (JWT, Rate Limiting, Caching)
- ✓ **Observability & Debugging** (Structured Logging, Distributed Tracing)
- ✓ **Testing & CI/CD Readiness** (Coverage, Automation, Deployment Strategy)

Submission Guidelines

- Submit the project as a **public Git repository**.
- Include a **README.md** with:
 - Setup instructions
 - Assumptions made
 - Possible future enhancements.

Reference API

- [Frankfurter API Docs](#)

- Base URL: <https://api.frankfurter.app/>
- Example: <https://api.frankfurter.app/latest>